Université Gustave Eiffel

Laboratoire d'Informatique Gaspard Monge

Space Invader

Computer Vision

Auteurs : Badredin BELACHHEB & Janelle TOURNEUR & Jeremy SARRI & Lisa PIN

Encadrant : Florian VALADE

Champs-sur-Marne — Année universitaire 2025–2026

Table des matières

Computer Vision : Space Invaders **1**

1 Collecte des données 1

2 Bases 2

2.1 Neurone artificiel 2

2.2 Perceptron 2

3 Convolutional Neural Network 4

3.1 Architecture du CNN 5

3.2 Convolution 5

3.3 Classification 8

4 Modèle 9

4.1 choix modèle : YOLO 9

4.2 Code `train_model_yolo` 9

4.3 Entraînement du modèle 10

Computer Vision : Space Invaders

Nous avons choisi d'entraîner un réseau de neurones convolutif (CNN), car ce modèle est adapté au traitement d'images. Notre première méthode consiste à travailler directement sur l'image entière, sans reconnaissance unique de la main.

1 Collecte des données

Le script `collect_dataset.py` permet de créer notre dataset de photos qui servira à l'entraînement du modèle. Il active la webcam et enregistre l'image entière en la classant dans des dossiers selon le geste effectué et selon un split *train/validation*. L'utilisateur contrôle la collecte au clavier :

- Appuyer sur une touche entre `0` et `5` pour sélectionner une classe (`NONE`, `LEFT`, `RIGHT`, `FIRE`, `ENTER`, `QUIT`)
- Appuyer sur `c` pour activer ou désactiver la capture (5 images par seconde)
- Appuyer sur `v` pour basculer entre le mode *train* et le mode *validation*
- Appuyer sur `q` pour quitter le programme

Comme le CNN sera entraîné sur toute l'image, nous avons choisi des gestes très distincts afin de limiter les confusions entre les classes :

- `LEFT` : paume de la main gauche ouverte, avec les doigts collés ;
- `RIGHT` : paume de la main droite ouverte, avec les doigts écartés ;
- `FIRE` : poing de la main droite fermé ;
- `ENTER` : deux paumes ouvertes ;
- `QUIT` : croix avec les bras.

Nous avons ajouté une classe `NONE` où il n'y a pas de gestes à reconnaître, pour éviter que le CNN classe des images même lorsque l'utilisateur ne fait rien.

2 Bases

2.1 Neurone artificiel

Mathématiquement, un neurone est une fonction

$$f_\omega : \mathbb{R}^n \to \mathbb{R}, \qquad x \mapsto \sigma(Wx + b)$$

Le vecteur d'entrée $x = (x_1, \ldots, x_n)$ représente les caractéristiques du problème. Chaque entrée est associée à un poids w_i qui mesure l'importance de cette variable dans la décision du neurone.

La première étape consiste à calculer une combinaison linéaire des entrées :

$$z = Wx + b = \sum_{i=1}^{n} w_i x_i + b$$

Le terme b est appelé *biais* et permet de décaler la fonction de décision.

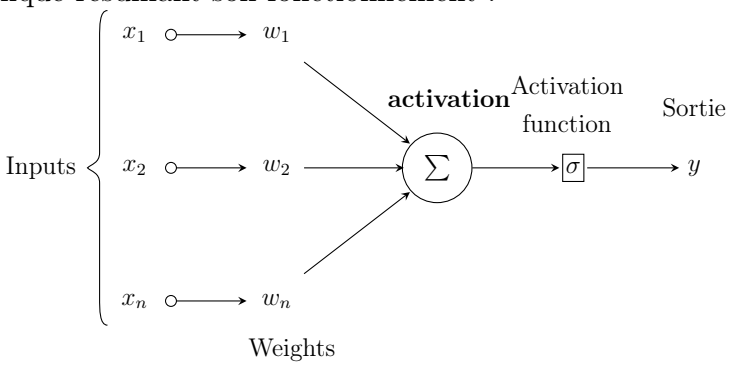
Ensuite, cette quantité z est transformée par une fonction d'activation σ. Cette fonction introduit une non-linéarité dans le modèle, ce qui permet aux réseaux de neurones d'approximer des relations complexes entre les variables.

Finalement, la sortie du neurone est donnée par

$$y = \sigma(z)$$

Ce mécanisme simple constitue l'unité de base des réseaux de neurones profonds, où plusieurs neurones sont empilés en couches afin d'apprendre des représentations de plus en plus abstraites des données.

Voici un graphique résumant son fonctionnement :



2.2 Perceptron

Un perceptron [?] est un réseau de neurones composé de deux couches :

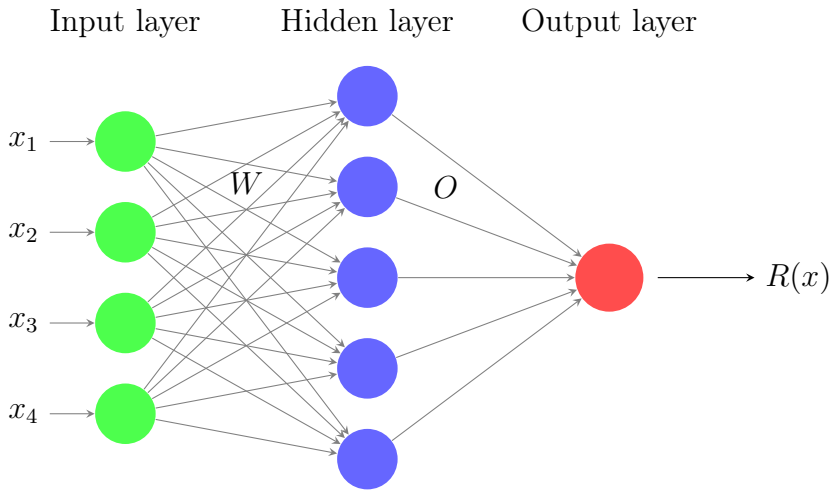


FIGURE 1 – Perceptron with $n = 4$, $k = 5$ and $p = 1$

1. une couche cachée dont tous les neurones partagent la même fonction d'activation non linéaire, généralement une sigmoïde,
2. une couche de sortie linéaire.

Le modèle est caractérisé par plusieurs paramètres structurels :

— n : le nombre de variables d'entrée,

— k : le nombre de neurones dans la couche cachée,

— p : le nombre de neurones dans la couche de sortie.

Les paramètres du réseau sont regroupés dans deux matrices :

— $W \in \mathcal{M}_{k,n}$ qui contient les poids reliant l'entrée à la couche cachée,

— $O \in \mathcal{M}_{p,k}$ qui contient les poids reliant la couche cachée à la couche de sortie.

Pour un vecteur d'entrée $x \in \mathbb{R}^n$, le réseau applique la transformation suivante :

$$R(x) = O\, s(Wx)$$

où $s : \mathbb{R} \to \mathbb{R}$ est une fonction sigmoïde appliquée composante par composante.
Le calcul se déroule donc en deux étapes :

1. une transformation linéaire Wx suivie de l'application de la fonction d'activation s, produisant les activations de la couche cachée,
2. une combinaison linéaire de ces activations via la matrice O pour produire la sortie finale.

Les matrices W et O constituent les paramètres du réseau, appelés *poids*, qui seront appris à partir des données lors de la phase d'entraînement.

3 Convolutional Neural Network

Un réseau neuronal convolutif s'inspire de la nature, car la connectivité entre les neurones artificiels ressemble à l'organisation du cortex visuel animal. L'un des principaux cas d'usage est la reconnaissance d'image. Les réseaux convolutifs apprennent plus rapidement et ont un meilleur taux d'erreur. Dans une moindre mesure, on les utilise aussi pour l'analyse vidéo. Ce type de réseau est aussi utilisé pour le traitement naturel du langage. Les modèles CNN sont très efficaces pour l'analyse sémantique, la modélisation de phrase, la classification ou la traduction.

Les CNN désignent une sous-catégorie de réseaux de neurones et sont à ce jour un des modèles de classification d'images réputés être les plus performant. Leur mode de fonctionnement est à première vue simple : l'utilisateur fournit en entrée une image sous la forme d'une matrice de pixels.

Celle-ci dispose de 3 dimensions :

— Deux dimensions pour une image en niveaux de gris.

— Une troisième dimension, de profondeur 3 pour représenter les couleurs fondamentales (Rouge, Vert, Bleu).

Contrairement à un modèle MLP (Multi Layers Perceptron) classique qui ne contient qu'une partie classification, l'architecture du Convolutional Neural Network dispose en amont d'une partie convolutive et comporte par conséquent deux parties bien distinctes :

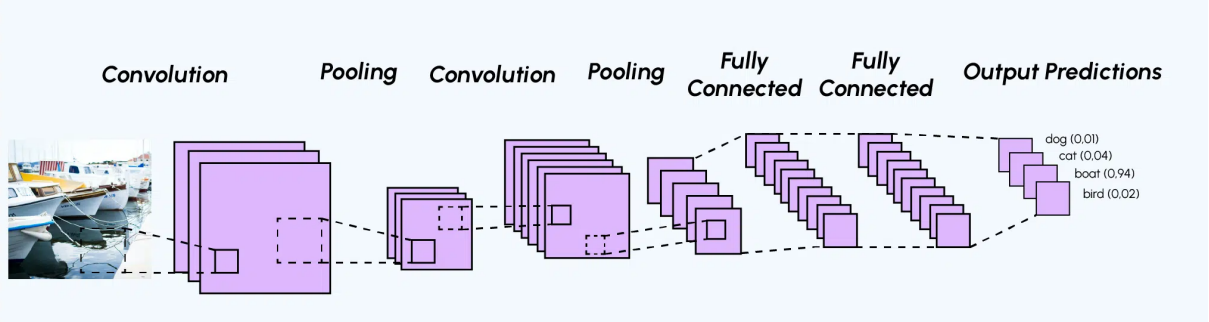
Une partie convolutive : Son objectif final est d'extraire des caractéristiques propres à chaque image en les compressant de façon à réduire leur taille initiale. En résumé, l'image fournie en entrée passe à travers une succession de filtres, créant par la même occasion de nouvelles images appelées cartes de convolutions. Enfin, les cartes de convolutions obtenues sont concaténées dans un vecteur de caractéristiques appelé code CNN. Une partie classification : Le code CNN obtenu en sortie de la partie convolutive est fourni en entrée dans une deuxième partie, constituée de couches entièrement connectées appelées perceptron multicouche (MLP pour Multi Layers Perceptron). Le rôle de cette partie est de combiner les caractéristiques du code CNN afin de classer l'image. Contrairement à un modèle MLP classique, l'ajout en amont de la partie convolutive permet d'obtenir en sortie une « carte de caractéristiques » ou « code CNN » (matrice de pixels située à droite dans l'exemple) dont les dimensions sont plus petites que celles de l'image initiale ce qui va avoir l'avantage de diminuer grandement le nombre de paramètres à calculer dans le modèle.

3.1 Architecture du CNN

Généralement, l'architecture d'un Convolutional Neural Network peut être décrit ainsi, en deux couches que l'on ajoute successivement :

— Couche de convolution (CONV) : Le rôle de cette première couche est d'analyser les images fournies en entrée et de détecter la présence d'un ensemble de features. On obtient en sortie de cette couche un ensemble de features maps.

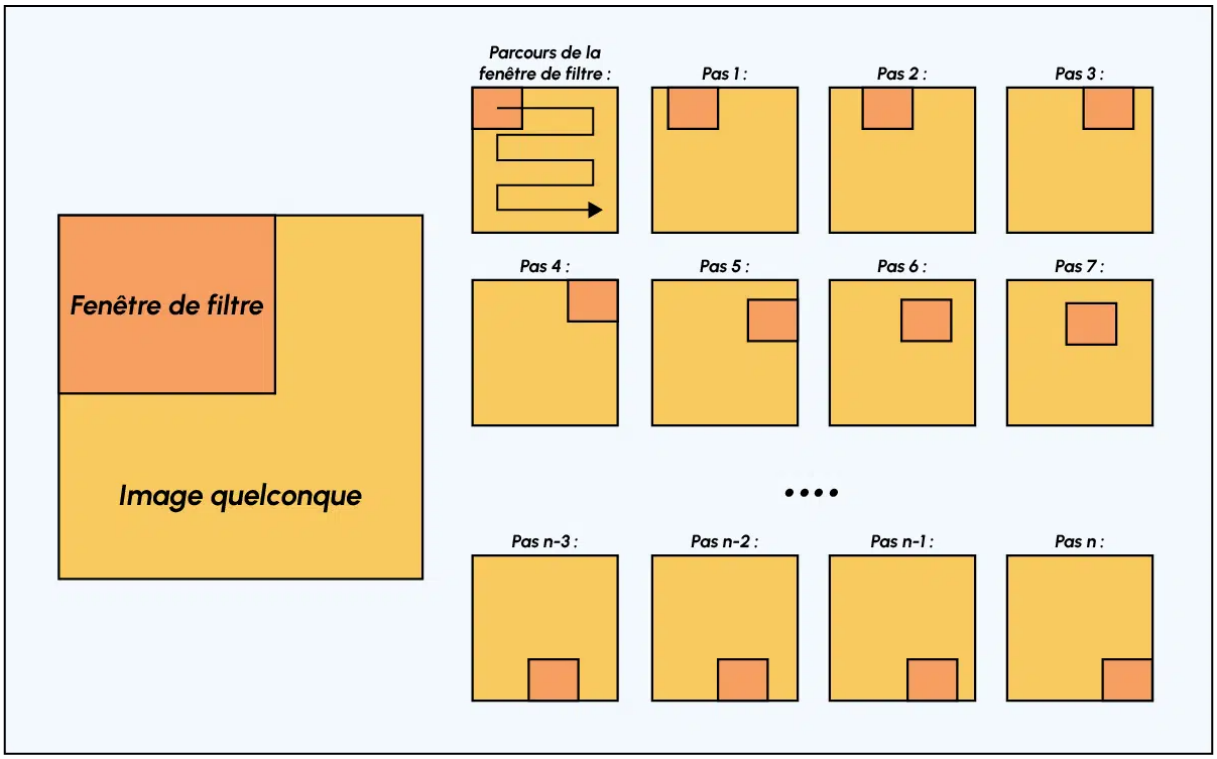
— Couche de Pooling (POOL) : La couche de Pooling est une opération généralement appliquée entre deux couches de convolution. Celle-ci reçoit en entrée les features maps formées en sortie de la couche de convolution et son rôle est de réduire la taille des images, tout en préservant leurs caractéristiques les plus essentielles. Parmi les plus utilisés, on retrouve le max-pooling mentionné précédemment ou encore l'average pooling dont l'opération consiste à conserver à chaque pas, la valeur moyenne de la fenêtre de filtre.



3.2 Convolution

La convolution est une opération mathématique simple généralement utilisée pour le traitement et la reconnaissance d'images. Sur une image, son effet s'assimile à un filtrage dont voici le fonctionnement :

1. Dans un premier temps, on définit la taille de la fenêtre de filtre située en haut à gauche.
2. La fenêtre de filtre, représentant la feature, se déplace progressivement de la gauche vers la droite d'un certain nombre de cases défini au préalable (le pas) jusqu'à arriver au bout de l'image.
3. À chaque portion d'image rencontrée, un calcul de convolution s'effectue permettant d'obtenir en sortie une carte d'activation ou feature map qui indique où est localisées les features dans l'image : plus la feature map est élevée, plus la portion de l'image balayée ressemble à la feature.

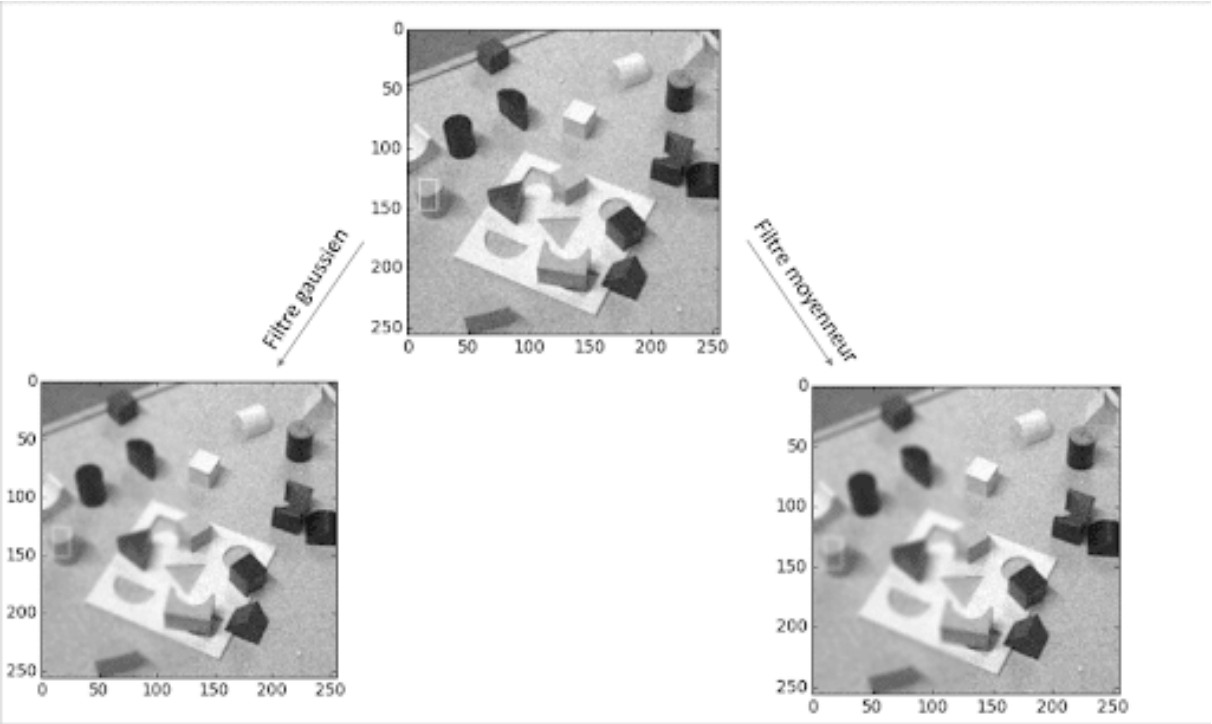


Exemple de convolution :

Lors de la partie convolutive d'un Convolutional Neural Network, l'image fournie en entrée passe à travers une succession de filtres de convolution. Par exemple, il existe des filtres de convolution fréquemment utilisés et permettant d'extraire des caractéristiques plus pertinentes que des pixels comme la détection des bords (filtre dérivateur) ou des formes géométriques. Le choix et l'application des filtres se fait automatiquement par le modèle.

Parmi les filtres les plus connus, on retrouve notamment le filtre moyenneur (calcule pour chaque pixel la moyenne du pixel avec ses 8 proches voisins) ou encore le filtre gaussien permettant de réduire le bruit d'une image fournie en entrée :

Voici un exemple des effets de ces deux différents filtres sur une image comportant un bruit important (on peut penser à une photographie prise avec une faible luminosité par exemple). Toutefois, un des inconvénients de la réduction du bruit est qu'elle s'accompagne généralement d'une réduction de la netteté :



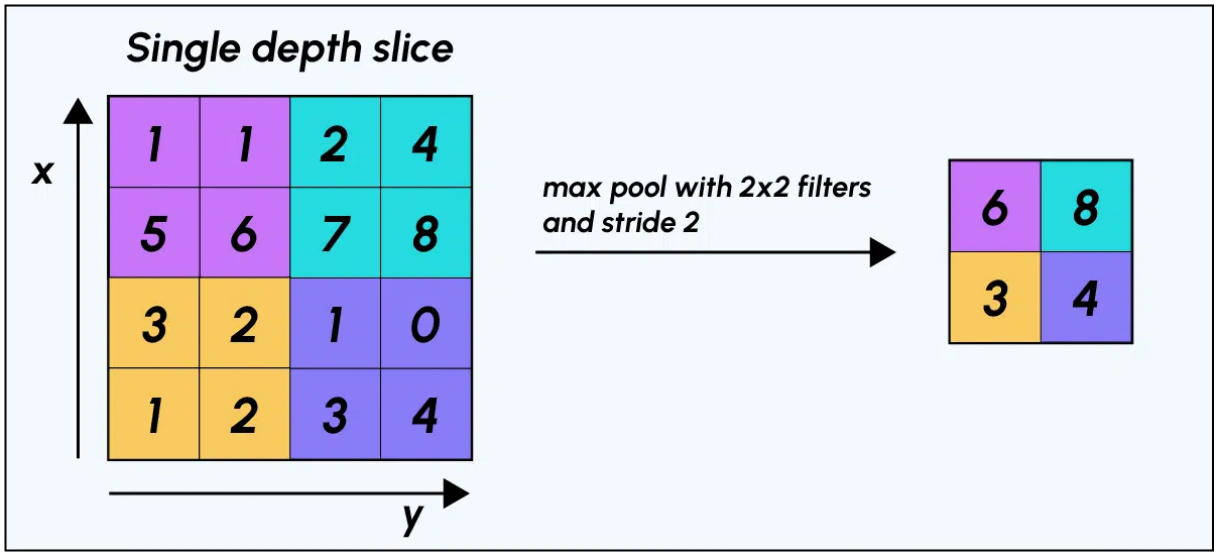
Outre sa fonction de filtrage, l'intérêt de la partie convolutive d'un CNN est qu'elle permet d'extraire des caractéristiques propres à chaque image en les compressant de façon à réduire leur taille initiale, via des méthodes de sous-échantillonnage tel que le Max-Pooling.

Max-Pooling

Le Max-Pooling est un processus de discrétisation basé sur des échantillons. Son objectif est de sous-échantillonner une représentation d'entrée (image, matrice de sortie de couche cachée, etc.) en réduisant sa dimension. De plus, son intérêt est qu'il réduit le coût de calcul en réduisant le nombre de paramètres à apprendre et fournit une invariance par petites translations [1].

1. Si une petite translation ne modifie pas le maximum de la région balayée, le maximum de chaque région restera le même et donc la nouvelle matrice créée restera identique

Exemple :



La fenêtre de filtre se déplace de deux pixels vers la droite (stride/pas = 2) et récupère à chaque pas l'argmax correspondant à la valeur la plus grande parmi les 4 valeurs de pixels.

3.3 Classification

L'objectif du CNN est d'attribuer à chaque échantillon de données une étiquette décrivant sa classe d'appartenance.

L'algorithme que les perceptrons utilisent pour mettre à jour leurs poids (ou coefficients de réseaux) s'appelle la rétropropagation du gradient de l'erreur

```python
import numpy as np
import matplotlib.pyplot as plt

def gradient_descent(f, grad, x0, alpha, n_iter=100):
    x = np.array(x0, dtype=float)
    traj = [x.copy()]
    for i in range(n_iter):
        x = x - alpha *grad(x)
        traj.append(x.copy())
    return np.array(traj)
```

Listing 1 – Algorithme de descente de gradient

4 Modèle

4.1 choix modèle : YOLO

Dans un premier essai, nous avons choisi d'utiliser le modèle YOLO (You Only Look Once), plus particulièrement la version YOLOv8 qui est spécifique à la classification d'images/la reconnaissance de gestes.
L'avantage de ce modèle est qu'il traite l'image dans son intégralité et non en plusieurs étapes. Il fonctionne par grille i.e. l'image est découpée selon un quadrillage où chaque cellule prédit si un objet s'y trouve ou non. Sur chaque cellule, le modèle YOLO va prédire, simultanément, les bounding boxes, un score de confiance ainsi que la probabilité des classes.
Le modèle YOLO à l'avantage d'être rapide quant à la détection d'image, ce qui est primordial pour jouer au jeu Space Invaders, il est cependant moins précis sur la localisation exacte des contours. Il permet également une détection multiple, i.e. la détection dans notre cas des deux mains pour les commandes `ENTER` ou `QUIT` par exemple.

4.2 Code `train_model_yolo`

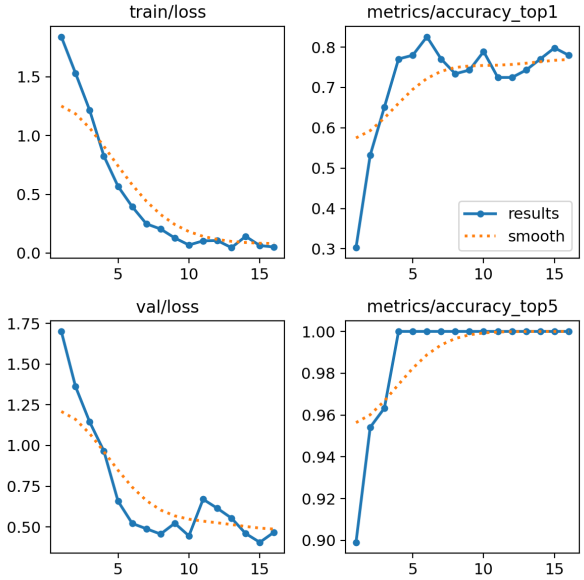
Le code d'entraînement du modèle est le code nommé "`train_model_yolo`". Ce code s'entraîne sur des data, que nous avons nous même recueillies grâce au code "`collect_dataset`". Ce code nous a permis de créer une banque de photos sur laquelle le modèle va pouvoir s'entraîner.
Il comptabilise dans un premier temps le nombres photos prises pour entraîner ("train") et valider ("val") le modèle.

```
Dossier validation : val/

Classe          Train      Val
------------------------------
ENTER              26        8
FIRE               18       21
LEFT               35       17
NONE               36       15
QUIT               25       24
RIGHT              51       24
------------------------------
TOTAL             191      109

Dataset YOLO deja prepare dans model_output\yolo_dataset

Chargement du modele YOLOv8n-cls (nano, ~6MB)...
Entrainement (50 epochs max, early stopping patience=10)...
```

Notre dataset considéré n'est pas parfaitement équilibré (64% de données d'entraînement et 36% de données de validation, loin du standard 80% 20%) mais le modèle peut quand même s'entraîner.
Le modèle analyse des millier de fois les photos du sous-dossier "train" afin de s'entraîner à reconnaître les gestes (position des mains) et ensuite il considère les photos du sous-dossier "val" qu'il ne connaît pas pour vérifier s'il a su détecter correctement les gestes.

4.3 Entraînement du modèle

Nous pouvons voir si le modèle est bien entraîné grâce à des courbes d'apprentissages et des comparaisons de photos.



i) Le graphique *train-loss* (:erreur du modèle) i.e. la courbe d'erreur sur les images d'entraînement : elle décroît rapidement et fini près de 0.1, ce qui signifie que le modèle à bien mémorisé les gestes.

ii) Le graphique *val-loss* qui correspond à la courbe d'erreurs sur les images de validation décroît également, jusqu'à 0.5 puis devient stable, on voit donc que le modèle ne fait pas de sur-apprentissage.

iii) Le graphique *metrics-accuracy_ top1* (accuracy = le pourcentage de bonne réponse) atteint les 80% de bonnes réponses.

iv) Le graphique *metrics-accuracy_ top5* n'est pas super intéressante puisqu'elle dit qu'on se trouve dans le top 5 des bonnes réponses mais il y a seulement 6 possibilités (`LEFT, NONE, QUIT...` donc c'est peu surprenant..

Notre modèle YOLO, pendant sa phase d'entraînenemt, à enregistré des images qui nous permettent de voir comment il apprend.

L'image de gauche correspond aux **vraies** commandes, il n'y a donc aucune erreur sur les photos, elles correspondent à "l'objectif" du modèle. L'image de droite correspond au résultat de notre modèle. On constate quelques erreurs, mais avec les courbes plus hautes on peut tout de même conclure que le modèle à fournit un travail relativement correct.
Enfin, notre code `train_model_yolo`, pour s'entraîner au mieux, s'auto crée des contraintes :

Ce qui lui permet d'anticiper d'éventuels changements de conditions (luminosité, cadrage ..) de sorte à être plus robuste.
Il aurait fallut un autre code afin de pouvoir jouer au jeu une fois le modèle entraîné mais nous avons décidé de changer d'approche.